

Part I: Classification

We tested five different classifiers on the dataset. In this report, I want to compare the performance of the different classifiers. The data is not linearly separable; it has a checkerboard structure. A scatter plot of the dataset in figure 1 also shows that the points in Class 0 and Class 1 follow a "checkerboard-like" pattern. This means the classes are not linearly separable, and we cannot split them with a straight line, and this means that the complex decision boundaries are needed for classifying them accurately.

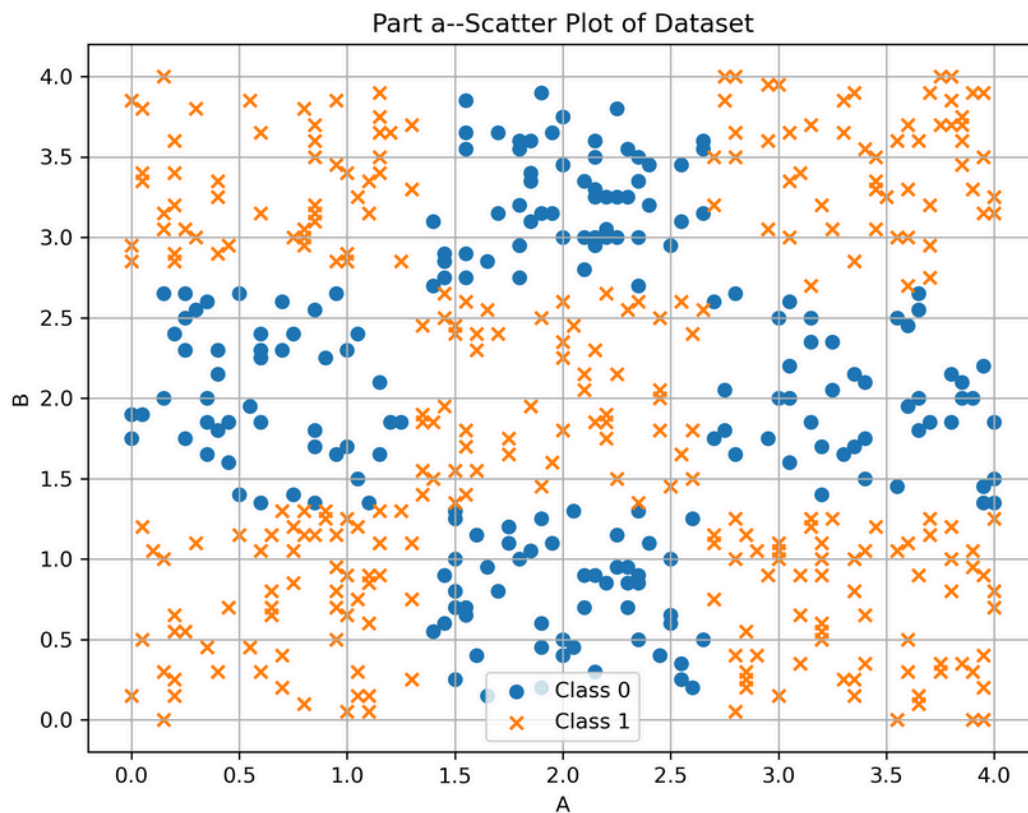
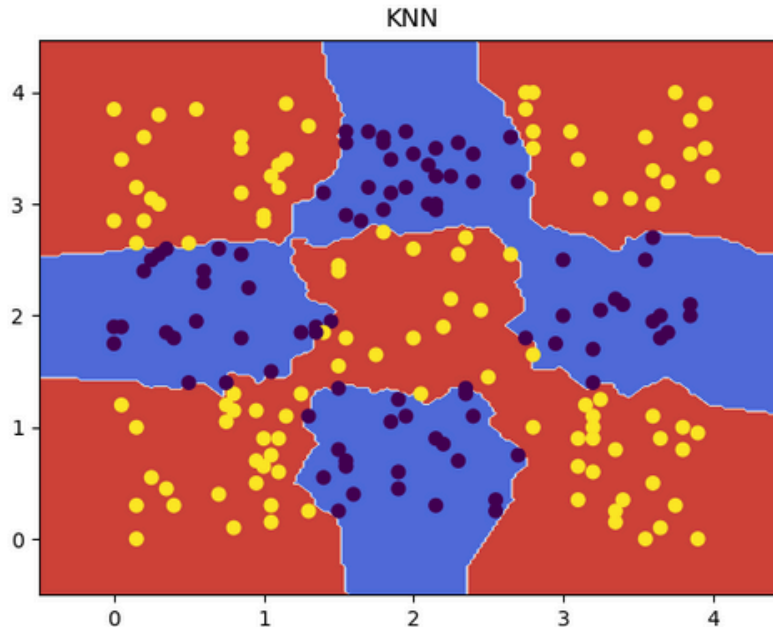


Figure 1

K-Nearest Neighbors



K-Nearest Neighbors algorithm (KNN) creates small regions to effectively separate class 1 and class 2. The decision boundary is effectively capturing the shape of the data distribution (figure 1). But I think the jagged decision boundary maybe sometimes make it sensitive to noise.

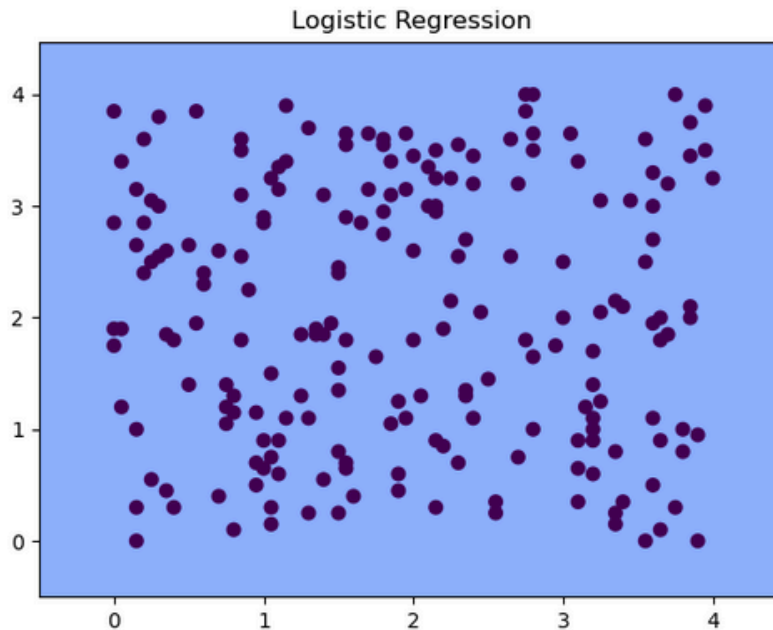
- **Performance:**

- **Training Score: 0.93.** This means KNN model correctly classifies 93% of the points in the training dataset.
- **Testing Score: 0.925.** This means KNN model performs well on unseen data, and captures the patterns in the data well. Its accuracy is almost the same as on the training set.

- **Best Parameters:**

- **leaf_size = 5:** It's maximum number of points that can be stored in each leaf node (subset).
- **n_neighbors = 3:** KNN uses 3 nearest neighbors to classify a point.

Logistic Regression



In this plot, the entire background is filled with one color blue, meaning the model predicts all points as a single class. So, the model can not effectively separate the data for different classes. I think this is because Logistic Regression assumes a linear relationship (straight line) between features and classes, and if the data distribution is complex (like here with chessboard-like), the algorithm has difficulty to draw a meaningful boundary and capture the complex patterns in the data. As a result, this model misclassifies many data points.

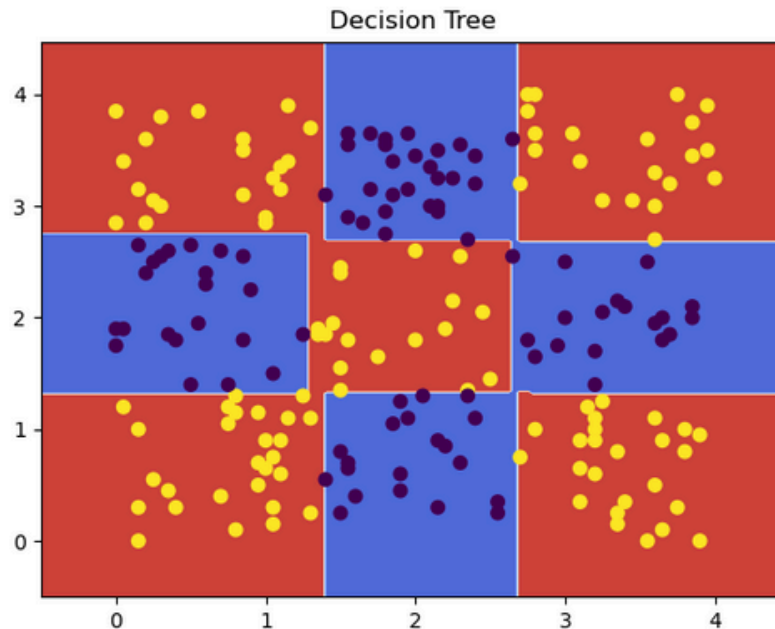
- **Performance:**

- **Training Score: 0.6067.** The model correctly classifies about 60.67% of the points in the training set. This indicates that the model is not capturing the data's patterns effectively.
- **Testing Score: 0.565.** The model's performance on unseen data is even lower, showing that it doesn't generalize well. It indicates that this algorithm is not the best choice for this task.

- **Best Parameters:**

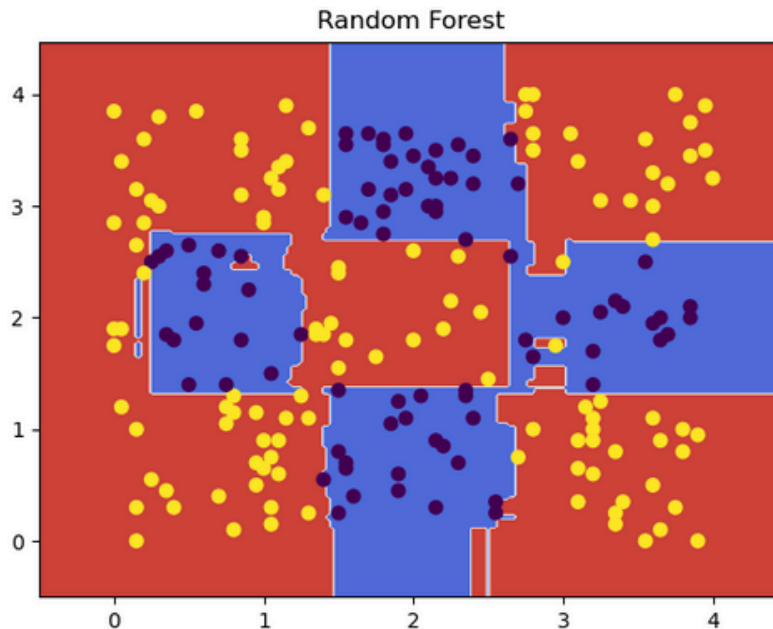
- **C = 0.1:** It is the regularization parameter that is used to prevent overfitting.

Decision Tree



- In this plot, the boundaries are horizontal or vertical because Decision Trees split the dataset based on feature values at specific thresholds to maximize the accuracy of classification. This was the best classifier! It learned the data perfectly and gave excellent results for unseen data.
- **Performance:**
 - **Training Score: 0.9733.** It shows the model is achieving a great accuracy on the training dataset, meaning it has learned the training patterns well.
 - **Testing Score: 0.995.** The model generalizes very great to unseen data, as shows by the high testing accuracy.
- **Best Parameters:**
 - **max_depth = 7:** This parameter limits the maximum depth of the tree.
 - **min_samples_split = 2:** A value of 2 means the tree will split as long as a node contains at least two samples. It allows for splits to be fine grained.

Random Forest



In this plot, the decision boundary is more complex in comparison to the Decision Tree, because Random Forest combines the predictions of multiple decision trees, which helps it generalize better to new data. It performed a little worse than Decision Tree on testing data because of better generalization. However, it still captures the pattern in the data very well.

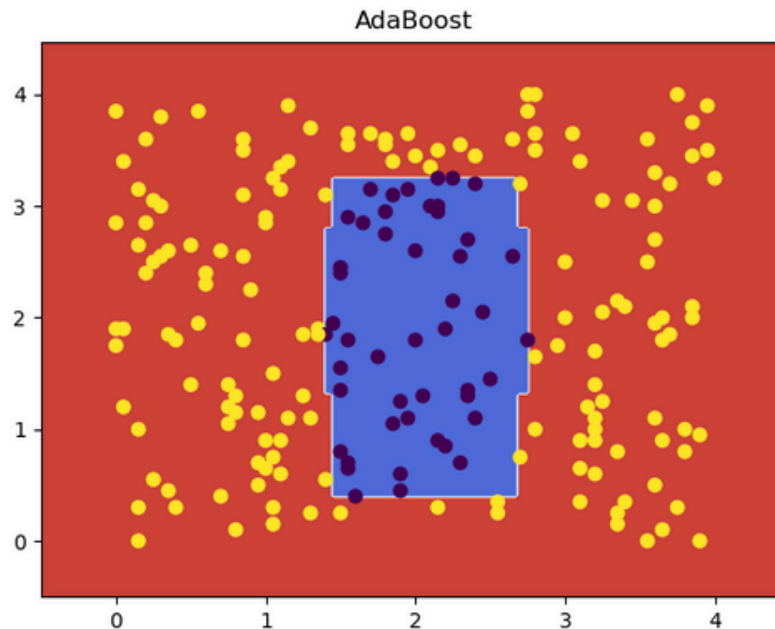
- **Performance:**

- **Training Score: 0.944.** The model performs well on the training data.
- **Testing Score: 0.945.** The testing accuracy is very close to the training accuracy, indicating that the model generalizes well to unseen data.

- **Best Parameters:**

- **max_depth = 5:** This limits the depth of each individual decision tree.
- **min_samples_split = 2:** This is the minimum number of samples required to split a node in a decision tree.

AdaBoost



In this plot, decision boundary is very simple, with rectangular shapes. The simple boundary reflects the limitations of using weak learners, and the need for more estimators to improve accuracy. This means that AdaBoost may not be suitable for this dataset because it has failed to capture the complex patterns in the data here and it shows that the dataset requires very complex decision boundaries. It did better than Logistic Regression but significantly lower than other methods like Decision Tree or KNN still.

- **Performance:**

- **Training Score: 0.6133.** The model performs not great on the training data, meaning it is capturing only simple patterns.
- **Testing Score: 0.63.** The testing accuracy is slightly better than the training accuracy, indicating the model generalizes moderately well to unseen data.

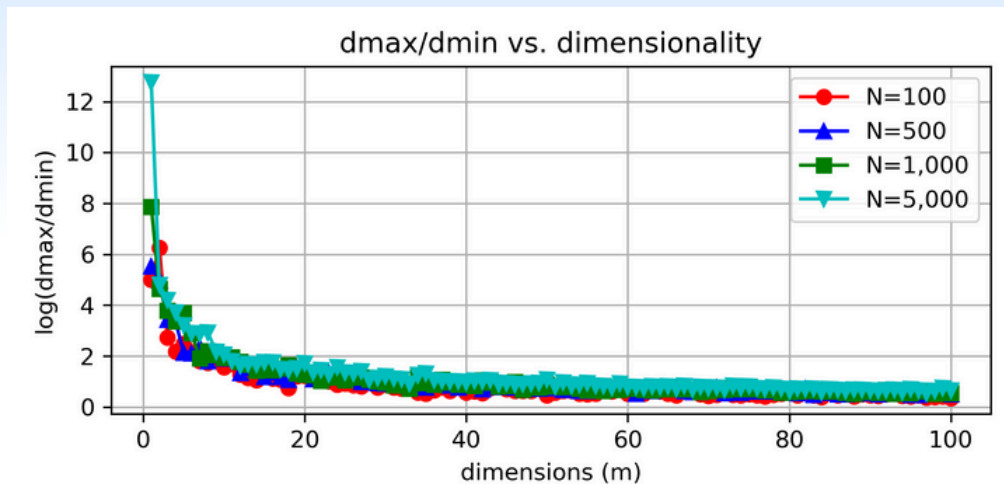
- **Best Parameters:**

- **n_estimators = 60:** This indicates the model uses 60 weak classifiers (like decision stumps) to combine into the final AdaBoost model.

Conclusion

- Decision Tree performed the best overall, achieving the highest testing score and effectively capturing the data's structure.
- Random Forest came second, offering slightly lower performance but better generalization.
- KNN also performed very well and was a close third, with a balanced score and flexible decision boundary.
- AdaBoost also performed poorly due to their simplicity and inability to handle the complex patterns in the dataset.
- Logistic Regression had the lowest scores among all classifiers. Its linear nature limits its ability to handle complex patterns in the data.

Part II (b): DMAX/DMIN Ratio vs. Dimensionality



The curse of dimensionality means that in high dimensions, points in a dataset become almost equally far from each other. This makes it hard for distance-based methods like KNN to work well. Here, we want to see how $D_{\max}/D_{\min}$ changes as the number of dimensions increases from $d=1$ to $d=100$. We created data using `make_classification` for different sample sizes. For each dimension, we chose a random query point, calculated distances to other points, and found $D_{\max}/D_{\min}$ that plotted on a logarithmic scale.

This plot shows how the logarithm of $D_{\max}/D_{\min}$ changes with the number of dimensions in the dataset, ranging from $d=1$ to $d=100$, for different sample sizes ($N=100, 500, 1000, 5000$). At first, in lower dimensions ($d < 20$), the $\log(D_{\max}/D_{\min})$ decreases very fast. It shows that in low dimensions, the difference between the nearest and farthest points is significant and has meaning, but as the number of dimensions increases, the differences become smaller, and the ratio stays the same.

After $d > 20$, the ratio levels off for all sample sizes. This stabilization shows that in higher dimensions, distances between points become nearly uniform, as predicted by the curse of dimensionality. Even when more dimensions are added, the ratio does not grow a lot, indicating that $D_{\max}/D_{\min}$ remains steady. Larger sample sizes ($N=1000$ and $N=5000$) produce smoother lines with less variability in $\log(D_{\max}/D_{\min})$ compared to smaller sample sizes like $N=100$. This is because larger datasets provide more consistent estimates of distances and reduce noise, I guess. However, the overall trend remains the same regardless of the sample size.

Part II (c) - High dimensionality: Curse or Blessing?

Most people think that having a lot of data is a bad thing. They say that too much data makes it hard to find patterns and make decisions. This is called the "curse of dimensionality." However, some researchers believe that having a lot of data can actually be a good thing. This is called the blessing of dimensionality.

High-dimensional spaces help in several ways. First, it makes it easier to sort different types of data. For example, in medical images, features like shape, color, and texture can help doctors tell apart healthy and sick tissue. High-dimensional data also helps find patterns in data. With more features, hidden relationships become easier to see. For example, in finance, many different numbers about the economy can show how things are related, leading to better predictions. Another advantage is that high dimensions can reduce overfitting which happens when a model becomes too specific to the training data and fails with unseen data. In high dimensions, models must generalize because there are so many data points and features. This makes predictions more accurate and improves performance. For example, in natural language processing, large language models trained on massive datasets can generalize better to new text, avoiding overfitting.

However, high dimensions also has challenges. One challenge is that it can be slow to process. Not all features are useful—some add noise and confuse the model. For example, using hair color as a feature to predict height does not help and makes the task harder. To solve these problems, researchers use techniques like dimensionality reduction, which removes unnecessary features but keeps important information. They also use feature selection to select only the most useful features. Regularization is another method that keeps models simple and focused on the meaningful parts of the data.

In real-world examples, high dimensions bring many benefits. Recommendation systems, like Netflix, use many features about users and movies to suggest content. In image recognition, high-dimensional data helps models detect objects or faces in photos. In language processing, high dimensions help computers understand the meanings of words and how they relate to each other.

Some research supports the blessing of high dimensions. For example, *Donoho and Tanner (2005)* showed that high-dimensional data can still be recovered even with noise. *Belkin and Niyogi (2003)* showed that high-dimensional data can help create accurate classifiers, even for complex data. These studies prove that high dimensions can be very helpful.

For further reading, the Wikipedia page on the curse and blessing of dimensionality provides a great introduction to this topic. Another useful resource is the article *A Few Useful Things to Know About Machine Learning*, which explains many principles related to high-dimensional spaces and their impact on learning algorithms.

In conclusion, high-dimensional data can seem difficult to handle, but it offers many benefits. It helps separate data, find patterns, and improve predictions. While high dimensions have challenges, techniques like dimensionality reduction and feature selection can overcome them. Machine learning systems that use high-dimensional data are often smarter, faster, and more accurate.

Part II (c) - High dimensionality: Curse or Blessing?

References:

- Wikipedia: Curse and Blessing of Dimensionality
- A Few Useful Things to Know About Machine Learning
- Donoho, D. L., & Tanner, J. (2005). Neighborliness of random linear codes and high dimensional geometry I. Proceedings of the IEEE, 93(1), 289-301.
- Belkin, M., & Niyogi, P. (2003). Laplacian eigenmaps for dimensionality reduction and data representation. Neural computation, 15(6), 1373-1396.